бюджетное профессиональное образовательное учреждение Вологодской области
«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность **09.02.07** «Информационные системы и программирование»

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

ПП по ПМ.03 РЕВЬЮИРОВАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ

Выполнил студент 3 курса группы ИС-____

_____

подпись _____

место практики _____
наименование юридического лица, ФИО ИП

Период прохождения:

с «___» _____ 2026 г.

по «___» _____ 2026 г.

Руководитель практики от предприятия
должность_____

_____

подпись_____

Руководитель практики от
техникума: Материкова А.А.

_____

Оценка:_____

«___» _____2026 года

МП

г. Череповец

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ 4
1.1 Организационная структура предприятия 4
1.2 Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов 5
1.3 Должностные инструкции ИТ-специалистов предприятия 6
2 РЕВЬЮИРОВАНИЕ ПРОГРАММНЫХ ПРОДУКТОВ 7
2.1 Ревьюирование программного кода в соответствии с технической документацией 7
2.2 Измерение характеристик компонент программного продукта 8
2.3 Исследование созданного программного кода с использованием специализированных программных средств 8
2.4 Сравнительный анализ программных продуктов и средств разработки 9
3 ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ 10
3.1 Краткое описание разработанного программного продукта 10
3.2 Анализ средств разработки 11
3.3 Разработка структуры и архитектуры приложения 11
3.4 Разработка пользовательского интерфейса и загрузка изображений 13
3.5 Реализация функций обработки изображений 14
3.6 Реализация сохранения результата и дополнительных функций 15
3.7 Автоматизированное тестирование 16
3.8 Измерение характеристик программного продукта 16
3.8.1 Сравнение общего времени выполнения операций 17
3.8.2 Сравнение размеров файлов 17
3.9 Подготовка документации проекта 18
3.10 Итог работы 19
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 21
ПРИЛОЖЕНИЕ А 22

ВВЕДЕНИЕ

Производственная практика ПП.03 является важным этапом профессиональной подготовки, позволяющим закрепить теоретические знания и получить практические навыки в области разработки, анализа и тестирования программного обеспечения. В ходе практики изучаются особенности создания программных продуктов, применения современных средств разработки и оценки характеристик готовых приложений.

В рамках выполненного задания было разработано настольное приложение ImageEditor на языке программирования C# с использованием технологии WPF. Приложение предназначено для обработки изображений и предоставляет пользователю возможность изменять их размер, повышать резкость, обнаруживать контуры и просматривать результаты обработки в графическом интерфейсе.

В процессе работы было проведено обратное проектирование приложения с использованием UML-диаграмм, изучена структура программного кода, реализованы основные функциональные возможности и подготовлены README и автоматизированные тесты. Также были выполнены измерения времени обработки изображений и исследованы размеры файлов сборки программы.

Цель практики — закрепить навыки разработки и анализа программного обеспечения, изучить характеристики созданного приложения и применить специализированные средства для проверки его работоспособности.

Для достижения поставленной цели были определены следующие задачи:

- изучить структуру и функциональные возможности приложения ImageEditor;
- провести обратное проектирование программного продукта с использованием UML-диаграмм;
- реализовать и проверить основные функции обработки изображений;
- исследовать программный код с использованием средств разработки и тестирования;
- измерить время выполнения операций обработки изображений;
- определить размеры файлов сборки и проанализировать полученные результаты;
- изучить применяемые инструменты и технологии разработки программного обеспечения.

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ

«Малленом Системс» — компания, деятельность которой связана с разработкой программного обеспечения и внедрением технических решений для заказчиков. По предоставленной информации, в работе компании участвуют как подразделения, занимающиеся программными продуктами и поддержкой пользователей, так и подразделения, обеспечивающие техническую, коммерческую, административную и организационную деятельность.

Компания также выполняет работы на площадках предприятий различных отраслей, связанные с установкой и наладкой оборудования. В связи с этим отдельное внимание уделяется вопросам промышленной безопасности, электробезопасности, охраны труда и применению средств защиты при выполнении соответствующих работ.

Официальная информация о компании и направлениях её деятельности размещена на сайте: https://www.mallenom.ru/company/about/.

1.1 Организационная структура предприятия

Согласно предоставленным материалам, организационная структура «Малленом Системс» является линейной: в подразделениях имеются руководители, а обязанности распределяются в соответствии с направлениями деятельности. Фамилии руководителей в настоящем описании не указываются.

Основные подразделения компании:

• Центр по развитию интеллектуальных систем, отдел разработки ПО — проектирование, разработка и оптимизация программного обеспечения для клиентов компании.

• Производственно-технический отдел — выполнение пусконаладочных работ на предприятиях, проектирование размещения оборудования и его установка.

• АУП (административно-управленческий персонал) — общее управление компанией, планирование деятельности, формирование стратегии развития и организация внешних коммуникаций.

• Группа маркетинга — разработка маркетинговой стратегии, продвижение бренда и продуктов, организация внутреннего и внешнего PR.

• Коммерческий отдел — взаимодействие с заказчиками, продажа продуктов и поиск новых клиентов.

• Отдел технической поддержки и контроля качества — помощь пользователям, проверка программного обеспечения и выявление ошибок и проблем.
• Отдел акселерационных и образовательных программ — разработка и проведение обучающих курсов по машинному зрению и языкам программирования.
• Отдел кадров — подбор, адаптация и сопровождение сотрудников, ведение кадрового документооборота.
• Юридический отдел — работа с документами и договорными обязательствами в соответствии с законодательством.
• Бухгалтерия — ведение бухгалтерского учёта и финансово-экономической деятельности.
• ОХР (общественно-хозяйственные рабочие) — поддержание порядка, уборка и обслуживание служебных помещений.

Для темы практики особенно важны отдел разработки программного обеспечения, а также отдел технической поддержки и контроля качества. Первый занимается созданием и совершенствованием программных продуктов, второй участвует в проверке программного обеспечения и выявлении ошибок.

1.2 Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов

По предоставленным сведениям, в компании установлен пятидневный рабочий график: с понедельника по пятницу, с 09:00 до 18:00. В организации предусмотрен специалист по охране труда, который проводит вводные инструктажи для новых сотрудников и практикантов. Также он занимается выдачей пропусков для инженеров, выполняющих пусконаладочные работы.

Согласно материалам компании, 20 сентября 2018 года была проведена специальная оценка условий труда. В предоставленной информации указано, что рабочие места, на которых установлены вредные производственные факторы, отсутствуют.

При этом сотрудники проходят обучение по промышленной безопасности, электробезопасности, охране труда, применению средств защиты и оказанию первой помощи в специализированном учебном центре. Для направлений промышленной безопасности и электробезопасности предусмотрена последующая аттестация в Северо-Западном управлении Ростехнадзора в Вологодской области. Такая подготовка связана с тем, что часть работ компании выполняется на промышленных площадках заказчиков.

При работе за компьютером ИТ-специалистам необходимо соблюдать установленные правила охраны труда, поддерживать порядок на рабочем месте,

использовать исправное оборудование и сообщать ответственному лицу о выявленных неисправностях. Конкретные требования предприятия следует сверять с действующими локальными инструкциями и материалами вводного инструктажа.

1.3 Должностные инструкции ИТ-специалистов предприятия

В предоставленных материалах отсутствуют тексты должностных инструкций конкретных сотрудников. Поэтому ниже приведено обобщённое описание функций, соответствующих указанным в материалах направлениям работы; его необходимо уточнить по должностным инструкциям, если они будут предоставлены.

Специалисты отдела разработки ПО участвуют в проектировании, написании, изменении и оптимизации программного обеспечения. В рамках этих задач могут выполняться анализ требований, разработка программных модулей, исправление ошибок, подготовка технической документации и проверка корректности внесённых изменений.

Специалисты отдела технической поддержки и контроля качества помогают пользователям при возникновении проблем, проверяют работу программных продуктов и фиксируют обнаруженные дефекты. При тестировании важно воспроизводить ошибку, описывать условия её появления и передавать информацию разработчикам для исправления.

При ревьюировании программных модулей разработчики и специалисты по качеству оценивают соответствие кода требованиям, понятность структуры, наличие потенциальных ошибок и возможность дальнейшего сопровождения. Конкретный перечень обязанностей, полномочий и требований к квалификации определяется официальными должностными инструкциями организации.

2 РЕВЬЮИРОВАНИЕ ПРОГРАММНЫХ ПРОДУКТОВ

Ревьюирование программных продуктов представляет собой проверку программного кода и связанных с ним материалов с целью выявления дефектов, оценки соответствия требованиям и повышения сопровождаемости программного обеспечения. Проверка может выполняться вручную разработчиком или коллегами, а также с использованием автоматизированных инструментов.

Ревьюирование не заменяет тестирование: эти процессы дополняют друг друга. Анализ кода помогает обнаруживать потенциальные проблемы в реализации и структуре, а тестирование позволяет проверить поведение программы на выбранных входных данных и сценариях использования.

2.1 Ревьюирование программного кода в соответствии с технической документацией

Проверка программного кода должна опираться на требования проекта, техническое задание, описание интерфейсов, правила оформления кода и другие доступные документы. На первом этапе необходимо определить назначение проверяемого модуля и ожидаемое поведение. Затем сопоставляются требования и фактическая реализация.

Во время ревьюирования целесообразно проверять корректность алгоритмов, обработку ошибочных ситуаций, соблюдение соглашений об именовании, понятность функций и классов, отсутствие дублирования, а также соответствие кода принятой структуре проекта. Обнаруженные замечания следует формулировать конкретно: указывать место проблемы, ожидаемое поведение и возможное направление исправления.

Результаты проверки удобно фиксировать в виде списка замечаний с указанием файла, участка кода, описания проблемы и статуса исправления. Такой подход упрощает взаимодействие участников проекта и позволяет повторно проверить код после внесения изменений.

Фактическое ревьюирование конкретного модуля должно быть описано по результатам изучения исходного кода и технической документации проекта. В имеющихся материалах исходный код и техническое задание не представлены, поэтому конкретные дефекты и результаты проверки здесь не утверждаются.

2.2 Измерение характеристик компонент программного продукта

Измерение характеристик программных компонентов позволяет получить объективные сведения о размере, сложности и отдельных свойствах кода. Выбор показателей зависит от назначения программы, доступных инструментов и требований проекта.

Для анализа могут применяться следующие характеристики: количество строк кода; количество файлов, функций и классов; цикломатическая сложность; количество повторяющихся фрагментов; число замечаний, выявленных статическими анализаторами; результаты тестирования и покрытие кода тестами, если соответствующие инструменты и тесты имеются.

Полученные значения следует рассматривать в контексте проекта. Например, большое количество строк само по себе не означает низкое качество, а высокая цикломатическая сложность отдельных функций может указывать на необходимость более внимательного ревьюирования и дополнительных тестов. Для корректного сравнения компонентов важно использовать одинаковые правила подсчёта и сопоставимые версии исходного кода.

Численные результаты измерений в данном варианте отчёта не приводятся, поскольку исходный проект и результаты работы анализаторов в предоставленных файлах отсутствуют. Их следует добавить после проведения измерений.

2.3 Исследование созданного программного кода с использованием специализированных программных средств

Специализированные средства анализа помогают автоматизировать часть задач ревьюирования. Статические анализаторы проверяют исходный код без его выполнения и могут выявлять потенциальные ошибки, подозрительные конструкции, нарушения стиля и некоторые проблемы безопасности. Форматтеры обеспечивают единообразное оформление кода, а средства тестирования позволяют проверить работу программы на заданных сценариях.

При выборе инструмента необходимо учитывать используемый язык программирования, тип проекта, совместимость с системой сборки и возможность интеграции в рабочий процесс. Для получения полезного результата важно правильно

настроить правила анализа, изучить каждое существенное предупреждение и отличать реальные дефекты от ложных срабатываний.

Типовая последовательность исследования включает подготовку копии проекта, запуск инструмента, изучение отчёта, классификацию замечаний, проверку наиболее важных проблем вручную и повторный запуск анализа после исправлений. При документировании результатов указывают название и версию инструмента, проверенную версию проекта, настройки и обнаруженные проблемы.

Поскольку в приложенных материалах не указаны фактически использованные анализаторы и не представлены отчёты их работы, этот подраздел описывает общий порядок исследования. Сведения о реально применённых инструментах и результатах необходимо внести после выполнения соответствующего задания.

2.4 Сравнительный анализ программных продуктов и средств разработки

Сравнение средств разработки следует проводить по заранее определённым критериям, связанным с задачами проекта. К таким критериям могут относиться поддерживаемые языки программирования, удобство навигации по коду, возможности отладки, интеграция с системами контроля версий, наличие расширений, поддержка тестирования и статического анализа, требования к ресурсам и простота освоения.

Сравнительный анализ должен основываться на проверяемых характеристиках, документации разработчиков и результатах практического использования. Нельзя делать вывод о превосходстве одного средства только по количеству функций: важны потребности команды, тип проекта и доступная инфраструктура.

Для оформления результатов удобно составить таблицу, в которой перечислить сравниваемые средства, критерии и обнаруженные особенности. После заполнения таблицы формулируется вывод о том, какой инструмент лучше подходит для рассматриваемых задач и почему.

Конкретные продукты для сравнения в предоставленных материалах не названы. Поэтому окончательные названия инструментов, их версии и результаты практического сравнения следует добавить после согласования набора средств и выполнения проверки.

3 ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ

В ходе производственной практики был разработан настольный программный продукт ImageEditor, предназначенный для загрузки, обработки и сохранения изображений. Работа включала анализ средств разработки, создание структуры проекта, проектирование архитектуры, реализацию пользовательского интерфейса и функций обработки изображений, автоматизированное тестирование, измерение характеристик программного продукта и подготовку документации.

3.1 Краткое описание разработанного программного продукта

ImageEditor представляет собой настольное приложение для работы с изображениями. Программа позволяет загружать исходные файлы, просматривать изображения, отображать их основные характеристики, изменять размер, повышать резкость, обнаруживать контуры и сохранять обработанный результат.

Пользовательский интерфейс объединяет основные операции в одном окне, что упрощает работу с приложением. Параметры обработки задаются пользователем, после чего приложение формирует результат с учётом выбранных настроек.

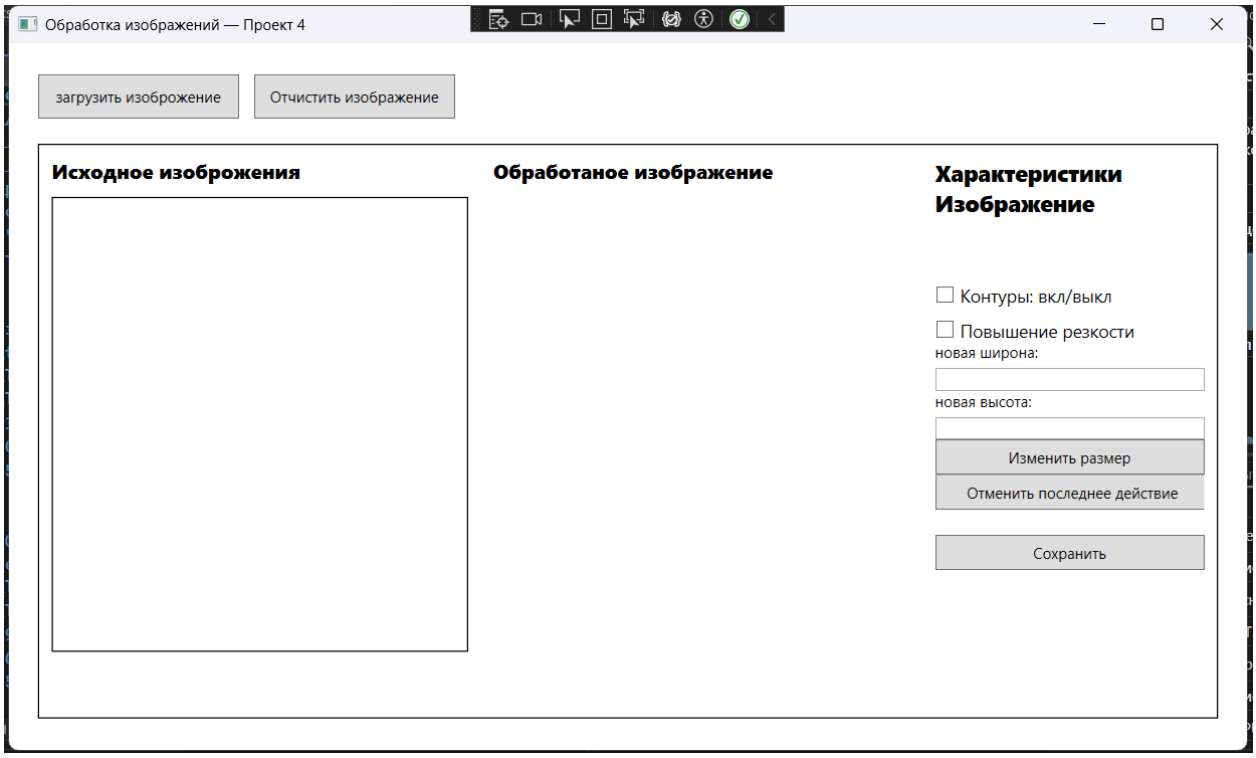


Рисунок 1 — Главное окно приложения ImageEditor

3.2 Анализ средств разработки

Для разработки приложения использовались язык программирования C#, платформа .NET 8, технология WPF, язык разметки XAML, библиотека ImageSharp, средство автоматизированного тестирования NUnit и среда разработки Microsoft Visual Studio.

C# применялся для реализации логики приложения, обработки пользовательских действий и взаимодействия между программными компонентами. Платформа .NET 8 обеспечивала выполнение приложения и использование необходимых библиотек. Технология WPF и язык XAML использовались для создания графического интерфейса и размещения элементов управления.

Библиотека ImageSharp применялась для загрузки и обработки изображений, включая повышение резкости, обнаружение контуров и изменение размеров. NUnit использовался для организации автоматизированных тестов библиотеки обработки изображений. Microsoft Visual Studio обеспечивала создание проектов, редактирование кода, сборку решения и запуск приложения.

Выбор перечисленных средств позволил разделить пользовательский интерфейс и обработку изображений, а также предусмотреть отдельный проект для проверки работоспособности программной логики.

3.3 Разработка структуры и архитектуры приложения

На начальном этапе было создано решение ImageEditorSolution в среде Microsoft Visual Studio. Оно включает три проекта: ImageEditor, ImageProcessor и ImageProcessor.Tests.

Проект ImageEditor отвечает за графический интерфейс и взаимодействие с пользователем. В нём реализованы загрузка изображений, настройка параметров обработки, отображение исходного и обработанного изображений, а также сохранение результата. Проект ImageProcessor содержит логику обработки изображений, сосредоточенную в классе ImageService. Проект ImageProcessor.Tests предназначен для автоматизированного тестирования функций библиотеки.

Разделение на отдельные проекты позволяет разграничить обязанности компонентов и упрощает сопровождение программного продукта. Пользовательский

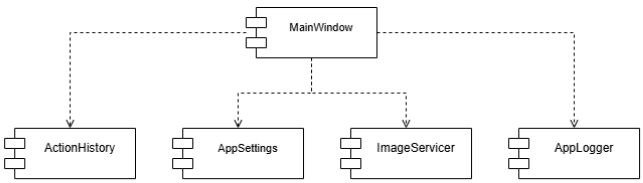
интерфейс взаимодействует с библиотекой обработки, а тесты проверяют её основные сценарии независимо от графического интерфейса.

Для описания структуры и взаимодействия компонентов были подготовлены архитектурная документация и UML-диаграммы. В документе «Архитектурные решения и технологии» описываются назначение модулей и используемые технологии. Диаграмма структуры модулей показывает состав решения и разделение его частей, а диаграмма последовательности отражает порядок взаимодействия компонентов при обработке изображения.

При проектировании приложения были приняты архитектурные решения, направленные на разделение обязанностей между программными компонентами. Пользовательский интерфейс реализован в проекте ImageEditor, операции обработки изображений вынесены в библиотеку ImageProcessor, а автоматизированные тесты размещены в отдельном проекте ImageProcessor.Tests.

Основным компонентом обработки является класс ImageService, предоставляющий метод ProcessImage для выполнения выбранных операций над изображением. Такой подход позволяет сосредоточить логику обработки в одном месте и вызывать её из пользовательского интерфейса с передачей необходимых параметров. Для обработки изображений используется библиотека ImageSharp, а взаимодействие с пользователем реализовано средствами WPF.

Отдельное выделение тестового проекта позволяет проверять работу библиотеки обработки независимо от графического интерфейса. Принятая структура упрощает сопровождение приложения, разделяет пользовательскую и функциональную части и делает архитектуру проекта более понятной.



Модуль	Описание
MainWindow	Модуль пользовательского интерфейса. Обеспечивает загрузку и отображение изображений, выбор фильтров, изменение размеров, сохранение результата и взаимодействие с другими модулями.
ImageService	Модуль обработки изображений из библиотеки ImageProcessor. Выполняет обнаружение контуров, повышение резкости, изменение размеров и получение характеристик изображения.
AppSettings	Модуль сохранения и восстановления настроек приложения. Хранит путь к последнему изображению, параметры фильтров, размеры изображения и размеры окна программы.
ActionHistory	Модуль ведения истории действий. Записывает даты, названия операций и их параметры в JSON-файл.
AppLogger	Модуль журналирования. Записывает события работы приложения и сообщения об ошибках в файл журнала.

Рисунок 2 — UML-диаграмма структуры модулей приложения ImageEditor

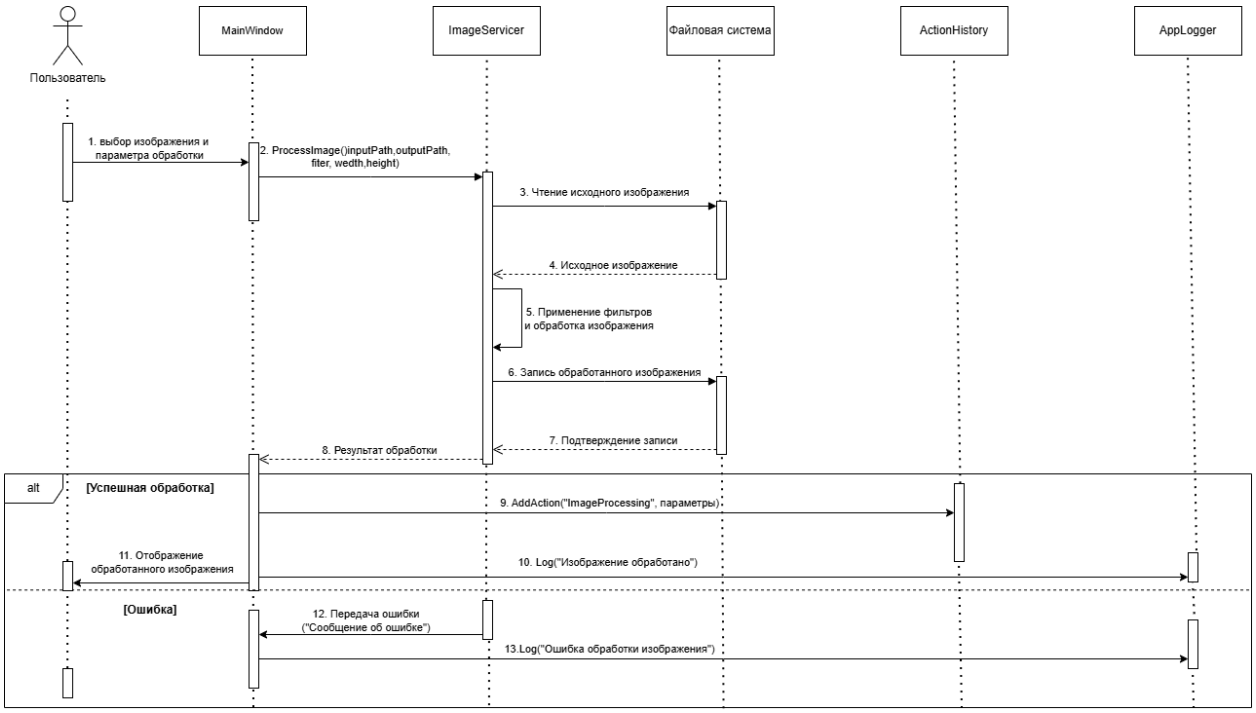


Рисунок 3 — Диаграмма последовательности обработки изображения

3.4 Разработка пользовательского интерфейса и загрузка изображений

Графический интерфейс приложения был реализован с использованием WPF и XAML. В главном окне предусмотрены области для просмотра исходного и обработанного изображений, отображение характеристик изображения, переключатели операций обработки, поля ввода ширины и высоты, а также кнопка сохранения результата.

Для загрузки исходного файла используется диалоговое окно выбора изображения. После выбора файла приложение получает путь к нему и загружает изображение для предварительного просмотра. Также отображаются основные характеристики исходного изображения: ширина, высота и формат.

Наличие двух областей просмотра позволяет сравнивать исходное изображение с результатом обработки. Отображение характеристик помогает пользователю проверить параметры исходного файла перед выполнением операций.

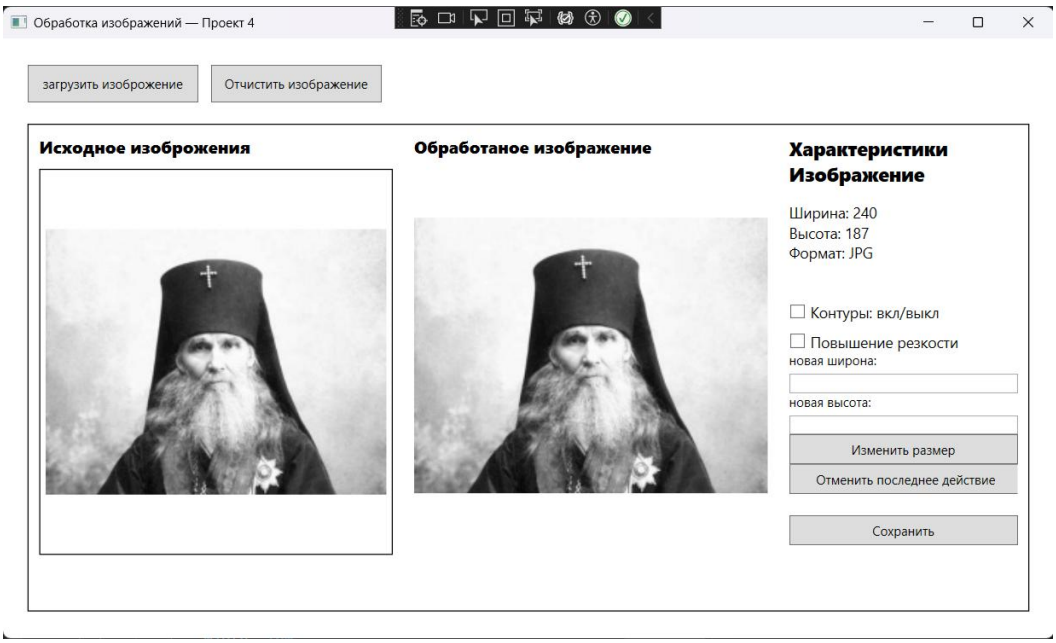


Рисунок 4 — Отображение исходного изображения и его характеристик

3.5 Реализация функций обработки изображений

Основные операции обработки реализованы в проекте ImageProcessor в классе ImageService. Для выполнения обработки используется метод ProcessImage, который принимает путь к исходному файлу, путь сохранения результата, параметры повышения резкости и обнаружения контуров, а также необязательные значения ширины и высоты.

Перед загрузкой изображения проверяется существование исходного файла. Если файл отсутствует, метод генерирует исключение FileNotFoundException. Если файл существует, изображение загружается, после чего к нему последовательно применяются выбранные операции.

Повышение резкости выполняется с помощью средств ImageSharp. Обнаружение контуров применяется при включённом соответствующем параметре. Изменение размера производится при наличии значений новой ширины и высоты. После завершения выбранных операций изображение сохраняется по указанному пути.

Использование общего метода обработки позволяет объединить несколько операций в одном алгоритме и применять их в зависимости от выбранных пользователем параметров.

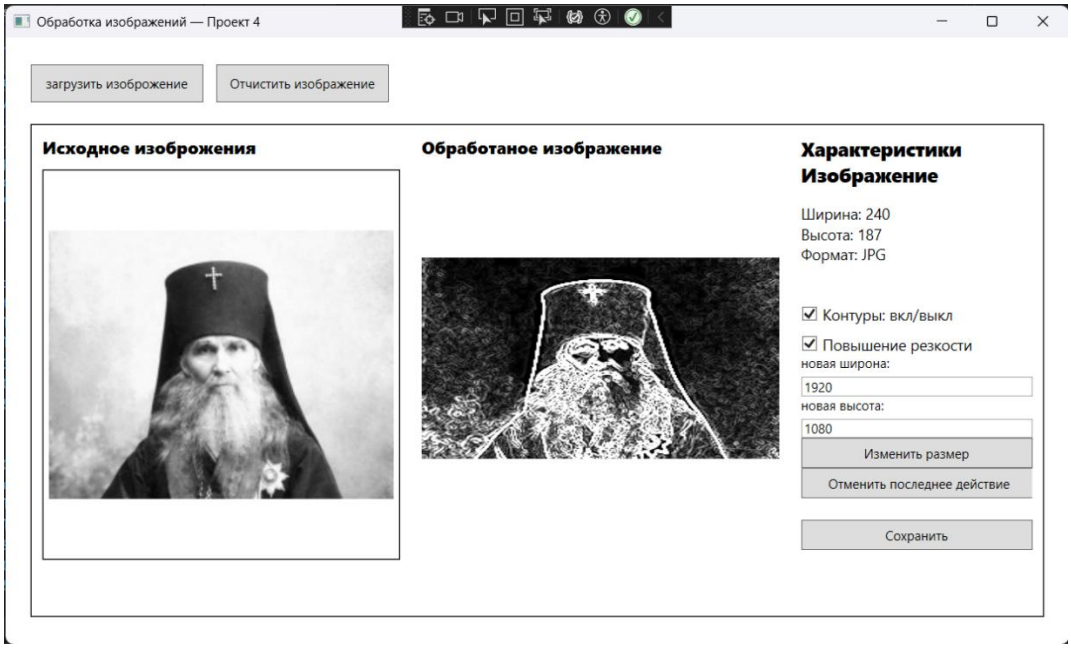


Рисунок 5 — Результат обработки изображения

3.6 Реализация сохранения результата и дополнительных функций

Для сохранения обработанного изображения в приложении предусмотрено диалоговое окно выбора имени и расположения файла. Пользователь может указать путь сохранения и имя итогового изображения. Результат сохраняется с учётом выбранных параметров обработки.

Помимо основных функций работы с изображениями, в проекте реализованы дополнительные механизмы, связанные с настройками приложения и обработкой действий пользователя. В коде главного окна предусмотрены методы сохранения настроек, обработки закрытия приложения, очистки изображения и работы с предыдущим состоянием.

Эти элементы расширяют возможности приложения и обеспечивают организацию логики пользовательского интерфейса.

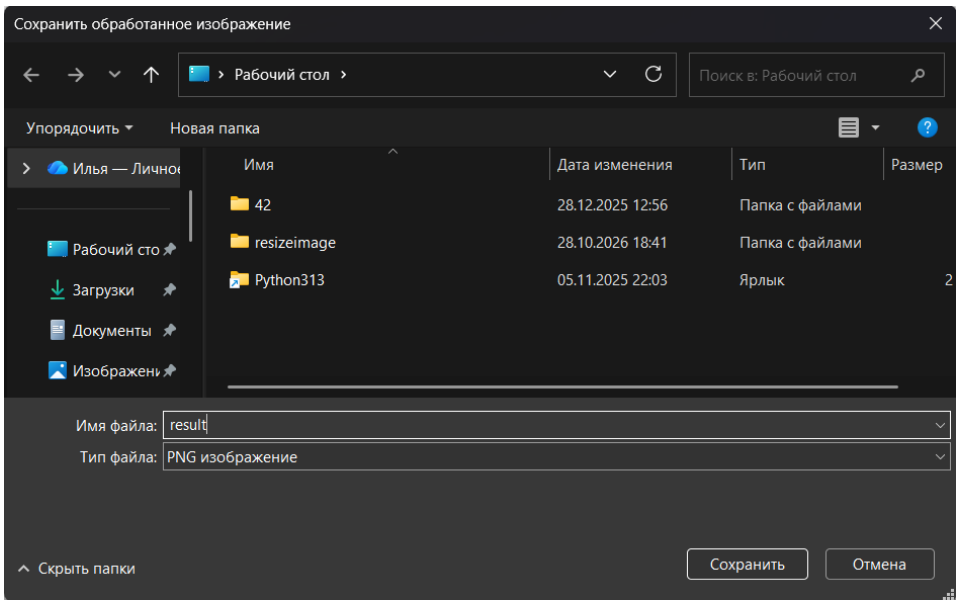


Рисунок 3.6 — Диалоговое окно сохранения обработанного изображения

3.7 Автоматизированное тестирование

Для проверки библиотеки обработки изображений был создан отдельный проект ImageProcessor.Tests с использованием NUnit. Тестирование предназначено для проверки основных сценариев работы программной логики.

Согласно документации проекта, тесты охватывают обработку изображения без дополнительных фильтров, изменение размеров, сохранение результата с применением повышения резкости и обнаружения контуров, а также поведение программы при отсутствии исходного файла.

Выделение тестов в отдельный проект позволяет проверять функции библиотеки независимо от пользовательского интерфейса. Это облегчает обнаружение ошибок в обработке файлов и выполнении основных операций.

3.8 Измерение характеристик программного продукта

Для оценки характеристик разработанного программного продукта были выполнены измерения времени выполнения операций и размеров файлов. Полученные результаты позволяют рассмотреть временные затраты при выполнении различных сценариев обработки и объём, занимаемый файлами приложения.

3.8.1 Сравнение общего времени выполнения операций

В ходе измерений фиксировалось время выполнения ранее выбранных сценариев. Для повышения наглядности результаты повторных измерений были сведены в таблицу «Сравнение общего времени». Она позволяет сопоставить полученные значения и оценить различия между сценариями.

Таблица 1 — Сравнение общего времени выполнения операций

Сценарий/запуск	1	2	3	4	5	Среднее
Без эффектов	9	9	9	9	9	9 мс
Только контуры	23	22	23	23	23	22,8 мс
Резкость + контуры	18	17	20	22	15	18,4 мс
Эффекты + изменение размера, сценарий A	138	138	141	139	140	139,2 мс
Эффекты + изменение размера, сценарий B	109	109	110	111	110	109,8 мс

По результатам сравнения можно определить, какие из измеренных сценариев занимали больше времени. При анализе следует учитывать условия проведения измерений и названия сценариев, указанные в исходной таблице. Это позволяет избежать ошибочного отнесения полученных значений к операциям, которые не соответствуют фактическим условиям тестирования.

3.8.2 Сравнение размеров файлов

Дополнительно были рассмотрены размеры файлов и объём, занимаемый выходными данными проекта. Для фиксации полученных характеристик была подготовлена таблица размеров файлов. Она используется для сравнения фактических значений, полученных при измерении.

Таблица 2 — Сравнение размеров файлов

№	Файл	Размер
1	ImageEditor.exe	148 КБ
2	ImageEditor.dll	22 КБ
3	ImageProcessor.dll	8 КБ
4	SixLabors.ImageSharp.dll	2544 КБ
Продолжение таблицы 2		

№	Файл	Размер
5	System.IO.Hashing.dll	55 КБ
6	ImageEditor.deps.json	3 КБ
7	ImageEditor.runtimeconfig.json	1 КБ
8	ImageEditor.pdb	23 КБ
9	ImageProcessor.pdb	13 КБ

В рамках отдельного измерения была проверена папка выходных файлов проекта ImageEditor. На момент проверки в ней находилось 661 файлов и 11 папки. Общий размер составлял 3 680 802 байт, или приблизительно 3,50МБ, а размер на диске — 4 472 832 байт, или приблизительно 4,26 МБ. Эти значения относятся к проверенной папке выходных файлов и не характеризуют размер всего решения ImageEditorSolution.

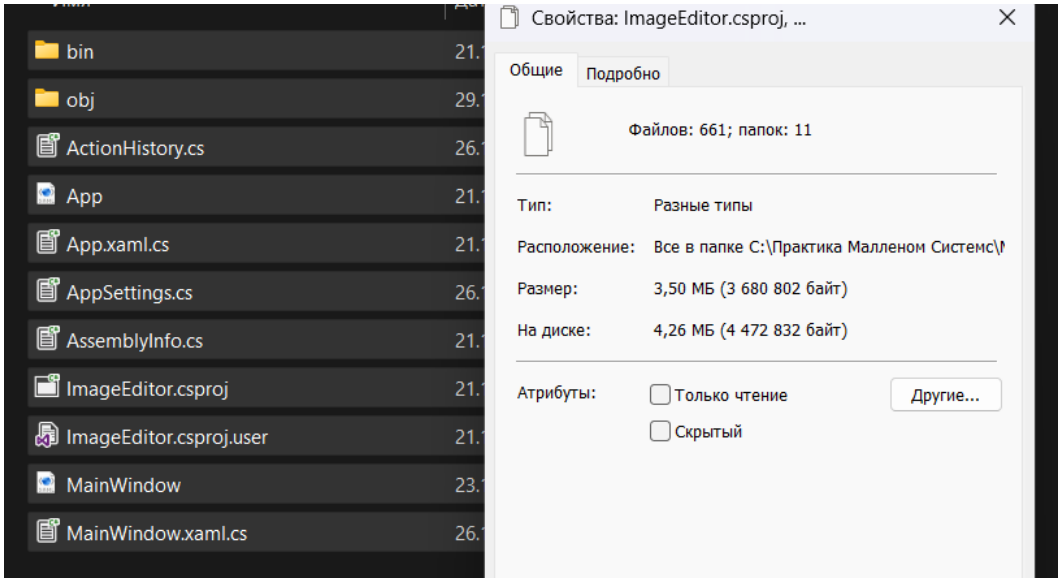


Рисунок 3.7 — Свойства папки выходных файлов проекта ImageEditor

3.9 Подготовка документации проекта

После реализации основных функций была подготовлена документация программного продукта. В файле README.md содержится краткое описание приложения, информация о структуре решения, основных возможностях и порядке запуска. Также в нём приводятся сведения о запуске автоматизированных тестов.

Дополнительно был подготовлен документ «Архитектурные решения и технологии», описывающий архитектуру приложения, назначение его основных компонентов и применяемые технологии. UML-диаграммы дополняют документацию графическим представлением структуры модулей и взаимодействия компонентов.

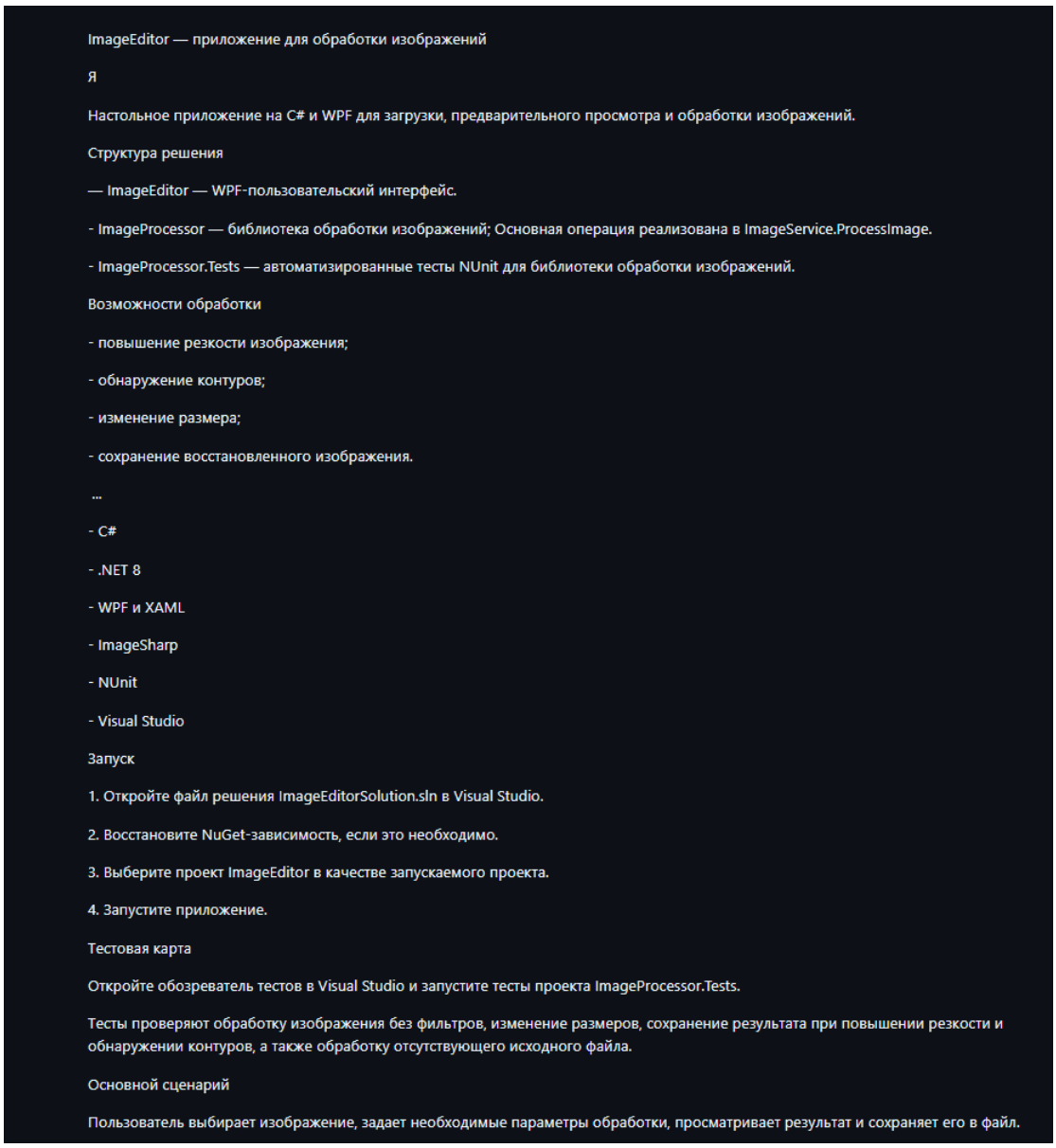


Рисунок 3.8 — Документация проекта в файле README.md

3.10 Итог работы

Таким образом, в ходе выполнения задания было разработано приложение ImageEditorSolution для загрузки, просмотра и обработки изображений. Реализованы повышение резкости, обнаружение контуров, изменение размеров и сохранение обработанных изображений. Проект разделён на пользовательский интерфейс, отдельную библиотеку обработки и модуль автоматизированного тестирования. Дополнительно реализованы вспомогательные механизмы сохранения настроек, ведения журнала событий и истории действий. Исходный код программы представлен в приложении А данного отчёта.

ЗАКЛЮЧЕНИЕ

В ходе прохождения производственной практики была выполнена работа по разработке настольного программного продукта ImageEditor, предназначенного для загрузки, обработки и сохранения изображений. В процессе выполнения заданий были изучены и применены средства разработки на языке C# с использованием платформы .NET 8, технологии WPF и библиотеки ImageSharp.

В рамках практики была создана структура решения, включающая проекты ImageEditor, ImageProcessor и ImageProcessor.Tests. Такое разделение позволило разграничить пользовательский интерфейс, логику обработки изображений и автоматизированное тестирование. Были реализованы основные функции приложения: загрузка изображений, отображение их характеристик, повышение резкости, обнаружение контуров, изменение размеров и сохранение обработанного результата.

Для описания структуры программного продукта были подготовлены архитектурные материалы и UML-диаграммы. Также была разработана документация, содержащая сведения о назначении приложения, его структуре, используемых технологиях и порядке запуска. Для проверки основных сценариев работы библиотеки был предусмотрен отдельный проект автоматизированных тестов на основе NUnit.

В ходе практики также проводилась оценка характеристик программного продукта, включая измерение времени выполнения операций и размеров файлов. Согласно полученным данным, папка проекта ImageEditor содержит 661 файл и 11 папок, её общий размер составляет 3 680 802 байта, а размер на диске — 4 472 832 байта.

В результате выполненной работы были закреплены практические навыки разработки приложений на C#, организации структуры программного решения, создания графического интерфейса, использования сторонних библиотек, подготовки технической документации и оценки характеристик программного продукта.

Таким образом, цели производственной практики были реализованы посредством разработки приложения ImageEditor и выполнения задач, связанных с проектированием архитектуры, реализацией функциональности, тестированием и документированием программного продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Learn. Документация по языку программирования C# [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 29.10.2026).
2. Microsoft Learn. Документация по платформе .NET 8 [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/core/whats-new/dotnet-8/overview (дата обращения: 29.10.2026).
3. Microsoft Learn. Документация по технологии Windows Presentation Foundation (WPF) [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 29.10.2026).
4. Six Labors. ImageSharp: документация библиотеки обработки изображений [Электронный ресурс]. — URL: https://docs.sixlabors.com/articles/imagesharp/ (дата обращения: 29.10.2026).
5. NUnit. Официальная документация средства автоматизированного тестирования [Электронный ресурс]. — URL: https://docs.nunit.org/ (дата обращения: 29.10.2026).
6. Microsoft. Visual Studio: официальная информация о среде разработки [Электронный ресурс]. — URL: https://visualstudio.microsoft.com/ (дата обращения: 29.10.2026).
7. Object Management Group. Unified Modeling Language (UML): спецификация языка визуального моделирования [Электронный ресурс]. — URL: https://www.omg.org/spec/UML/ (дата обращения: 29.10.2026).
8. Microsoft. Документация по языку программирования C# [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 29.10.2026).
9. Microsoft. Документация по WPF [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 29.10.2026).
10. Six Labors. Документация библиотеки ImageSharp [Электронный ресурс]. — URL: https://docs.sixlabors.com/ (дата обращения: 29.10.2026).

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ВСЕГО ПРОЕКТА

ФАЙЛ 1. MainWindow.xaml

```
<Window x:Class="ImageEditor.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:ImageEditor"
        mc:Ignorable="d"
        Title="Обработка изображений — Проект 4"
        Height="600"
        Width="1000">

    <Grid>
        <Button
            Content="загрузить изоброжение"
            Click="Button_Click"
            HorizontalAlignment="Left"
            VerticalAlignment="Top"
            Margin="23,25,0,0"
            Width="160"
            Height="35"
        />

        <Button
            Content="Отчистить изображение"
            Click="ClearImageButton_Click"
            HorizontalAlignment="Left"
            VerticalAlignment="Top"
            Margin="195,25,0,0"
            Width="160"
            Height="35"
        />

        <Border
            BorderBrush="Black"
            BorderThickness="1"
            Margin="23,80,23,25">

            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="3*"/>
                    <ColumnDefinition Width="3*"/>
                    <ColumnDefinition Width="2*"/>
```

```
</Grid.ColumnDefinitions>

<StackPanel Grid.Column="0" Margin="10">
  <TextBlock
    Text="Исходное изоброжения"
    FontSize="16"
    FontWeight="Black"
    Margin="0,0,0,10"
  />

  <Border
    BorderBrush="Black"
    BorderThickness="1"
    Padding="5">

    <Image
      x:Name="PreviewImage"
      Stretch="Uniform"
      Height="350"
    />
  </Border>
</StackPanel>

<StackPanel Grid.Column="1" Margin="10">
  <TextBlock
    Text="Обработаное изображение"
    FontSize="16"
    FontWeight="Black"
    Margin="0,0,0,10"
  />

  <Border>
    <Image
      x:Name="ProcessedImage"
      Stretch="Uniform"
      Height="350"
    />
  </Border>
</StackPanel>

<StackPanel Grid.Column="2" Margin="10">
  <TextBlock
    Text="Характеристики Изображение"
    FontSize="18"
    FontWeight="Black"
    TextWrapping="Wrap"
    Margin="0,0,0,15"
  />
```

```
<TextBlock
   x:Name="ImageInfoText"
   Text="Загрузите изображение"
   FontSize="14"
   TextWrapping="Wrap"
/>

<CheckBox
   x:Name="EdgesCheckBox"
   Content="Контуры: вкл/выкл"
   Margin="0,20,0,0"
   FontSize="14"
   Checked="EdgesCheckBox_Changed"
   Unchecked="EdgesCheckBox_Changed"
/>

<CheckBox
   x:Name="SharpenCheckBox"
   Content="Повышение резкости"
   Margin="0,10,0,0"
   FontSize="14"
   Checked="SharpenCheckBox_Changed"
   Unchecked="SharpenCheckBox_Changed"
/>

<TextBlock Text="новая широна:" Width="216"/>
<TextBox x:Name="WidthTextBox" Margin="0,5,0,0"/>

<TextBlock Text="новая высота:"/>
<TextBox x:Name="HeightTextBox" Margin="0,5,0,0"/>

<Button
   Content="Изменить размер"
   Click="ResizeButton_Click"
   Padding="8,5"
/>

<Button
   Content="Отменить последнее действие"
   Click="UndoLastAction_Click"
   Padding="10,5"
   Width="219"
/>

<Button
   Content="Сохранить"
   Click="SaveResult_Click"
   Margin="0,20,0,0"
   Padding="8,5"
```

```
                />
            </StackPanel>
        </Grid>
    </Border>
  </Grid>
</Window>
```

ФАЙЛ 2. MainWindow.xaml.cs

```
using ImageProcessor;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using Microsoft.Win32;

namespace ImageEditor
{
  public partial class MainWindow : Window
  {
    private string? _inputImagePath;

    private BitmapSource? _previousProcessedImage;
    private bool _previousEdgesEnabled;
    private bool _previousSharpenEnabled;
    private int? _previousWidth;
    private int? _previousHeight;

    private bool _isUndoing;

    public MainWindow()
    {
      InitializeComponent();

      Closing += MainWindow_Closing;

      _settings = AppSettings.Load();

      EdgesCheckBox.IsChecked = _settings.EdgesEnabled;
      SharpenCheckBox.IsChecked = _settings.SharpenEnabled;
```

```
    WidthTextBox.Text = _settings.NewWidth?.ToString() ?? "";
    HeightTextBox.Text = _settings.NewHeight?.ToString() ?? "";

    Width = _settings.WindowWidth;
    Height = _settings.WindowHeight;

    if (!string.IsNullOrWhiteSpace(_settings.LastImagePath))
        try
        {
            LoadImageFromPath(_settings.LastImagePath);
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                $"Не удалось обработать изображение.\nПричина: {ex.Message}",
                "Ошибка обработки",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }

    string appFolder = AppDomain.CurrentDomain.BaseDirectory;

    string logsFolder = System.IO.Path.Combine(appFolder, "logs");
    string configsFolder = System.IO.Path.Combine(appFolder, "configs");
    string outputFolder = System.IO.Path.Combine(appFolder, "output");

    Directory.CreateDirectory(logsFolder);
    Directory.CreateDirectory(configsFolder);
    Directory.CreateDirectory(outputFolder);
}

private readonly ImageService _imageService = new ImageService();
private AppSettings _settings = new AppSettings();
private readonly AppLogger _logger = new AppLogger();
private readonly ActionHistory _actionHistory = new ActionHistory();

private int? _newWidth;
private int? _newHeight;

private void Button_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog dialog = new OpenFileDialog();
    dialog.Title = "Выберите изображение";
    dialog.Filter = "Файлы изображений|*.jpg;*.jpeg;*.png;*.bmp;*.gif";

    bool? result = dialog.ShowDialog();

    if (result == true)
    {
```

```
        try
        {
            LoadImageFromPath(dialog.FileName);

            _logger.Log("Изображение загружено: " + dialog.FileName);
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                $"Не удалось обработать изображение.\nПричина: {ex.Message}",
                "Ошибка обработки",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }
}

private void EdgesCheckBox_Changed(object sender, RoutedEventArgs e)
{
    if (_isUndoing)
    {
        return;
    }

    SavePreviousState();

    _previousEdgesEnabled = EdgesCheckBox.IsChecked != true;

    UpdateProcessedImage();
}

private void SharpenCheckBox_Changed(object sender, RoutedEventArgs e)
{
    if (_isUndoing)
    {
        return;
    }

    SavePreviousState();
    _previousSharpenEnabled = SharpenCheckBox.IsChecked != true;
    UpdateProcessedImage();
}

private void SavePreviousState()
{
    if (ProcessedImage.Source is BitmapSource currentImage)
    {
        _previousProcessedImage = currentImage;
        _previousEdgesEnabled = EdgesCheckBox.IsChecked == true;
```

```
        _previousSharpenEnabled = SharpenCheckBox.IsChecked == true;
        _previousWidth = _newWidth;
        _previousHeight = _newHeight;
    }
}

public void UpdateProcessedImage()
{
    if (string.IsNullOrEmpty(_inputImagePath))
    {
        return;
    }

    try
    {
        string outputPath = System.IO.Path.Combine(
            System.IO.Path.GetTempPath(),
            $"image_result_{Guid.NewGuid()}.png");

        _imageService.ProcessImage(
            _inputImagePath,
            outputPath,
            EdgesCheckBox.IsChecked == true,
            SharpenCheckBox.IsChecked == true,
            _newWidth,
            _newHeight);

        BitmapImage processedBitmap = new BitmapImage();
        processedBitmap.BeginInit();
        processedBitmap.UriSource = new Uri(outputPath);
        processedBitmap.CacheOption = BitmapCacheOption.OnLoad;
        processedBitmap.EndInit();

        processedBitmap.Freeze();

        ProcessedImage.Source = processedBitmap;

        _logger.Log("Изображение обработано");

        _actionHistory.AddAction("image processing",
            new Dictionary<string, string>
            {
                { "Edges", (EdgesCheckBox.IsChecked == true).ToString() },
                { "Sharpen", (SharpenCheckBox.IsChecked == true).ToString() },
                { "Width", _newWidth?.ToString() ?? "Original" },
                { "Height", _newHeight?.ToString() ?? "Original" }
            });
    }
    catch (Exception ex)
```

```
    {
        _logger.Log($"Ошибка обработки изображения: {ex.Message}");

        MessageBox.Show(
            $"Не удалось обработать изображение.\nПричина: {ex.Message}",
            "Ошибка обработки",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

private void UndoLastAction_Click(object sender, RoutedEventArgs e)
{
    if (_previousProcessedImage == null)
    {
        MessageBox.Show("Нет действия для отмены");
        return;
    }

    _isUndoing = true;

    try
    {
        ProcessedImage.Source = _previousProcessedImage;
        EdgesCheckBox.IsChecked = _previousEdgesEnabled;
        SharpenCheckBox.IsChecked = _previousSharpenEnabled;

        _newWidth = _previousWidth;
        _newHeight = _previousHeight;

        WidthTextBox.Text = _newWidth?.ToString() ?? "";
        HeightTextBox.Text = _newHeight?.ToString() ?? "";

        _previousProcessedImage = null;
        _isUndoing = false;
    }
    finally
    {
        _isUndoing = false;
    }
}

private void ResizeButton_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrEmpty(_inputImagePath))
    {
        MessageBox.Show("Сначало загрзите изображение!");
        return;
    }
```

```
    if (!int.TryParse(WidthTextBox.Text, out int width) ||
        !int.TryParse(HeightTextBox.Text, out int height) ||
        width <= 0 || height <= 0)
    {
        MessageBox.Show(
            "Введите ширину и высоту — положительными, целыми числами.");
        return;
    }

    SavePreviousState();

    _newWidth = width;
    _newHeight = height;
    UpdateProcessedImage();
}

public void SaveResult_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrEmpty(_inputImagePath))
    {
        MessageBox.Show("Сначало загрзите изображение!");
        return;
    }

    SaveFileDialog saveWindow = new SaveFileDialog();
    saveWindow.Title = "Сохранить обработанное изображение";
    saveWindow.Filter = "PNG изображение|*.png";
    saveWindow.FileName = "result.png";

    bool? result = saveWindow.ShowDialog();

    if (result == true)
    {
        try
        {
            _imageService.ProcessImage(
                _inputImagePath,
                saveWindow.FileName,
                EdgesCheckBox.IsChecked == true,
                SharpenCheckBox.IsChecked == true,
                _newWidth,
                _newHeight);

            MessageBox.Show("Изображение сохранено!");

            _logger.Log(
                "Изменённое изоброжение сохраненно:" + saveWindow.FileName);
```

```
            _actionHistory.AddAction("saving an image",
                new Dictionary<string, string>
                {
                    { "save Path", saveWindow.FileName },
                    { "Format", "PNG" },
                    { "Edges", (EdgesCheckBox.IsChecked == true).ToString() },
                    { "Sharpen", (SharpenCheckBox.IsChecked == true).ToString() },
                    { "Width", _newWidth?.ToString() ?? "Original" },
                    { "Height", _newHeight?.ToString() ?? "Original" }
                });
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                $"Не удалось обработать изображение.\nПричина: {ex.Message}",
                "Ошибка обработки",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }
}

private void SaveSettings()
{
    _settings.LastImagePath = _inputImagePath ?? "";
    _settings.EdgesEnabled = EdgesCheckBox.IsChecked == true;
    _settings.SharpenEnabled = SharpenCheckBox.IsChecked == true;

    _settings.NewWidth =
        int.TryParse(WidthTextBox.Text, out int width) ? width : null;

    _settings.NewHeight =
        int.TryParse(HeightTextBox.Text, out int height) ? height : null;

    _settings.WindowWidth = Width;
    _settings.WindowHeight = Height;
    _settings.Save();
}

private void MainWindow_Closing(
    object? sender,
    System.ComponentModel.CancelEventArgs e)
{
    SaveSettings();
}

private void LoadImageFromPath(string filePath)
{
    if (!File.Exists(filePath))
```

```
    {
        MessageBox.Show(
            "Выбранный файл не найден.",
            "Ошибка загрузки",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        return;
    }

    try
    {
        BitmapImage bitmap = new BitmapImage();
        bitmap.BeginInit();
        bitmap.UriSource = new Uri(filePath);
        bitmap.CacheOption = BitmapCacheOption.OnLoad;
        bitmap.EndInit();

        _inputImagePath = filePath;
        PreviewImage.Source = bitmap;

        int width = bitmap.PixelWidth;
        int height = bitmap.PixelHeight;

        string format = System.IO.Path.GetExtension(filePath)
            .TrimStart('.')
            .ToUpper();

        ImageInfoText.Text =
            $"Ширина: {width}\n" +
            $"Высота: {height}\n" +
            $"Формат: {format}\n";

        UpdateProcessedImage();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            $"Не удалось загрузить изображение.\nПричина: {ex.Message}",
            "Ошибка загрузки",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

private void ClearImageButton_Click(object sender, RoutedEventArgs e)
{
    _inputImagePath = null;
    _newWidth = null;
    _newHeight = null;
```

```
            PreviewImage.Source = null;
            ProcessedImage.Source = null;
            ImageInfoText.Text = "";

            WidthTextBox.Text = "";
            HeightTextBox.Text = "";

            EdgesCheckBox.IsChecked = false;
            SharpenCheckBox.IsChecked = false;

            _previousProcessedImage = null;

            _settings.LastImagePath = "";
            _settings.NewWidth = null;
            _settings.NewHeight = null;
            _settings.EdgesEnabled = false;
            _settings.SharpenEnabled = false;
            _settings.Save();

            _logger.Log("Изображение очищено");

            _actionHistory.AddAction(
                "image cleaning",
                new Dictionary<string, string>());
        }
    }
}
```

ФАЙЛ 3. ImageService.cs

```
using System.IO;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

namespace ImageProcessor
{
    public class ImageService
    {
        public void ProcessImage(
            string inputPath,
            string outputPath,
            bool detectEdges,
            bool sharpen,
            int? newWidth,
            int? newHeight)
        {
            if (!File.Exists(inputPath))
```

```
            {
                throw new FileNotFoundException(
                    "изоброжение не найдено",
                    inputPath);
            }

            using var image = Image.Load(inputPath);

            if (sharpen)
            {
                image.Mutate(x => x.GaussianSharpen());
            }

            if (detectEdges)
            {
                image.Mutate(x => x.DetectEdges());
            }

            if (newWidth.HasValue && newHeight.HasValue)
            {
                image.Mutate(
                    x => x.Resize(newWidth.Value, newHeight.Value));
            }

            image.Save(outputPath);
        }
    }
}
```

ФАЙЛ 4. AppLogger.cs

```
using System;
using System.IO;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ImageProcessor
{
    public class AppLogger
    {
        private readonly string _logFilePath;

        public AppLogger()
        {
            string appFolder = AppDomain.CurrentDomain.BaseDirectory;
            string logsFolder = Path.Combine(appFolder, "logs");
```

```
            Directory.CreateDirectory(logsFolder);

            _logFilePath = Path.Combine(logsFolder, "app.log");
        }

        public void Log(string message)
        {
            string logEntry =
                $"{DateTime.Now:yyyy-MM-dd HH:mm:ss} — {message}";

            File.AppendAllText(
                _logFilePath,
                logEntry + Environment.NewLine);
        }
    }
}
```

ФАЙЛ 5. AppSettings.cs

```
using System;
using System.IO;
using System.Text.Json;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.DirectoryServices;

namespace ImageEditor
{
    public class AppSettings
    {
        public string LastImagePath { get; set; } = "";

        public bool EdgesEnabled { get; set; }
        public bool SharpenEnabled { get; set; }

        public int? NewWidth { get; set; }
        public int? NewHeight { get; set; }

        public double WindowWidth { get; set; } = 1000;
        public double WindowHeight { get; set; } = 600;

        private static readonly string SettingsFilePath =
            Path.Combine(
                AppDomain.CurrentDomain.BaseDirectory,
                "configs",
```

```
            "settings.json");

        public void Save()
        {
            string? folder = Path.GetDirectoryName(SettingsFilePath);

            if (folder != null)
            {
                Directory.CreateDirectory(folder);
            }

            string json = JsonSerializer.Serialize(
                this,
                new JsonSerializerOptions { WriteIndented = true });

            File.WriteAllText(SettingsFilePath, json);
        }

        public static AppSettings Load()
        {
            if (!File.Exists(SettingsFilePath))
            {
                return new AppSettings();
            }

            string json = File.ReadAllText(SettingsFilePath);

            return JsonSerializer.Deserialize<AppSettings>(json)
                ?? new AppSettings();
        }
    }
}
```

ФАЙЛ 6. ActionHistory.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.Json;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Data;

namespace ImageEditor
{
    public class ActionRecord
    {
```

```
    public DateTime Date { get; set; }
    public string Operation { get; set; } = "";

    public Dictionary<string, string> Parameters { get; set; } = new();
}

public class ActionHistory
{
    private readonly string _historyFilePath;

    public ActionHistory()
    {
        string appFolder = AppDomain.CurrentDomain.BaseDirectory;

        _historyFilePath = Path.Combine(
            appFolder,
            "logs",
            "history.json");

        Directory.CreateDirectory(
            Path.GetDirectoryName(_historyFilePath)!);
    }

    public void AddAction(
        string operation,
        Dictionary<string, string> parameters)
    {
        List<ActionRecord> history = LoadHistory();

        history.Add(new ActionRecord
        {
            Date = DateTime.Now,
            Operation = operation,
            Parameters = parameters
        });

        string json = JsonSerializer.Serialize(
            history,
            new JsonSerializerOptions { WriteIndented = true });

        File.WriteAllText(_historyFilePath, json);
    }

    private List<ActionRecord> LoadHistory()
    {
        if (!File.Exists(_historyFilePath))
        {
            return new List<ActionRecord>();
        }
```

```
        try
        {
            string json = File.ReadAllText(_historyFilePath);

            return JsonSerializer.Deserialize<List<ActionRecord>>(json)
                ?? new List<ActionRecord>();
        }
        catch (JsonException)
        {
            System.Windows.MessageBox.Show(
                "Файл истории содержит некорректный JSON. История будет создана заново.",
                "Ошибка истории",
                System.Windows.MessageBoxButton.OK,
                System.Windows.MessageBoxImage.Warning);

            return new List<ActionRecord>();
        }
        catch (IOException ex)
        {
            System.Windows.MessageBox.Show(
                $"Не удалось прочитать файл истории: {ex.Message}",
                "Ошибка чтения",
                System.Windows.MessageBoxButton.OK,
                System.Windows.MessageBoxImage.Warning);

            return new List<ActionRecord>();
        }
    }
  }
}
```